

Dalhousie University
CSCI 3110 — Design and Analysis of Algorithms I
Fall 2024
Assignment 2

Distributed Tuesday, September 17 2024.

Due 11:59PM, Tuesday, September 24 2024.

Instructions:

1. Before starting to work on the assignment, make sure that you have read and understood policies regarding the assignments of this course, especially the policy regarding collaboration. <https://dal.brightspace.com/d2l/1e/content/338900/Home?itemIdentifier=D2L.LE.Content.ContentObject.ModuleC0-4457275>
2. To submit via Crowdmark, upload a **separate** image/PDF file (multiple pages are allowed) to each question. You may resubmit as often as necessary until the due date. A good strategy is to create an initial submission days in advance after you solve some problems, and make resubmissions later. More detailed submission information for Crowdmark can be found at:
<https://crowdmark.com/help/completing-and-submitting-an-assignment/>
3. If you submit a joint assignment, use Crowdmark to create a group and add group members before making submissions. The “What Will Students See” section of <https://crowdmark.com/help/creating-a-group-assignment> shows the steps and screenshots of creating a group and adding members. Note that, “to avoid overwriting submissions”, Crowdmark recommends “only one group member submits all the work”.
4. We encourage you to typeset your solutions using LaTeX. However, you are free to use other software or submit scanned handwritten assignments as long as **they are legible**. We have the right to take marks off for illegible answers.

Questions:

1. [12 marks] Order all the following twelve functions by order of growth from slowest to fastest. That is, find an arrangement $f_1(n), f_2(n), \dots, f_{12}(n)$ of these functions such that $f_1 = O(f_2(n)), f_2 = O(f_3(n)), \dots, f_{11} = O(f_{12}(n))$. Partition your list into groups such that two functions $f(n)$ and $g(n)$ are in the same group if and only if $f(n) = \Theta(g(n))$.

$$(\lg \lg n)^2, 20n + 5000, n^n, \lg(n!)$$

$$2^{2n}, n \ln n, n/\lg \lg n, (\ln n)^{1/1000}$$

$$n^{1.0001}, \lg(\lg^2 n), (2n)!, n/2^{100}$$

You are NOT required to justify your ordering.

2. [8 marks] Tom was a carpenter, and for some projects, he used long but thin and narrow wood stripes of the same size. When working on those wood strips, he cut them to different lengths and used each piece as a component of furniture or other wooden structures. It was often the case that, after cutting a stripe to obtain several pieces of specific sizes, the remaining piece was too short to be useful for the current project. He usually measured and recorded the length of such a piece, and stored it in his warehouse in case it was useful some day. Over time, he accumulated a large bundle of leftover strips of various lengths.

One day, Mike, a friend of Tom's, saw these leftover wood strips. Mike was a mathematics teacher, and he asked Tom to make a right triangle with three pieces of these strips which he could use to help him teach the students different types of triangles. To avoid creating even shorter leftover strips unnecessarily, Tom would like to first find out whether he has three pieces that happen to have the right lengths to form a right triangle. By Pythagoras' theorem, this means he needs to find out, among the lengths of these leftover strips he recorded, whether there are three lengths, a , b and c , that satisfies $a^2 + b^2 = c^2$.

For example, if the lengths of these pieces are 16, 40, 7, 23, 78, 9, 24, 41, 11, 65, then the answer is "yes", because three of these numbers, 40, 9, 41, satisfy $9^2 + 40^2 = 41^2$.

Your task is to use the brute-force approach to design an algorithm that solves this problem in $\Theta(n^3)$ time. You can assume that these lengths are stored in an input array of n numbers, and your algorithm need only output a "yes" or "no" answer. Your algorithm must check each triple of array entries at most once.

For this question, you must show why the running time of your algorithm is $\Theta(n^3)$; it is not sufficient to show that it is $O(n^3)$.

When analyzing the running time, you can make the following assumptions to simplify the analysis:

- (a) Each line of the pseudocode uses one unit of time if it uses a constant number of **primitive** operations only;
- (b) The number of times a loop test is executed is the same as the statement directly inside the loop is executed. For example, you can assume that the statement "for $i = 1$ to n " is executed n times.

By now you have learned enough to know the above simplifications (even simpler than those used in the analysis of insertion sort given in class) are reasonable.

Hint: when showing that the running time is indeed $\Theta(n^3)$, reviewing *Appendix C.1 Counting* in the textbook may help.

Important: In all of the assignments of this course, when you are asked to give an algorithm for a problem (including both question 2 and question 3 of this assignment), you are (unless otherwise indicated) expected to

- (i) describe the idea behind your algorithm in English;
- (ii) provide pseudocode;
- (iii) argue that your algorithm is correct; and
- (iv) analyze its running time.

Regarding requirement (iii): the level of details in your argument depends on your solution. If your solution makes use of non-trivial properties of the problem, then rigorous proofs of these properties are required. Otherwise, provide sufficient arguments to argue why your algorithm considers all possible cases and performs computation correctly.

For example, to argue the correctness of the algorithm `maxsubrangesum2` in Lecture 4, we can say: To show the correctness, we first observe that the two levels of loops in the algorithm enumerates all possible pairs of values of l and u that define valid subarrays $x[l..u]$. For each new value of l , line 3 of the pseudocode initializes sum to 0. Then line 5 makes use of the fact that the sum of the entries in subarray $x[l..u]$ is equal to the the sum of the entries in subarray $x[l..u - 1]$ plus $x[u]$. This correctly computes the sum of the entries in $x[l..u]$, because, when each iteration of the inner loop starts, the value in sum is equal to the sum of the entries of $x[l..u - 1]$. Thus our algorithm computes the sum of each subarray of x and this guarantees that we can find our desired result.

Regarding requirement (iv): Unless otherwise specified, show the steps of your analysis and present your result using big-Oh. The simplification assumptions used for Question 2 can also be used.

3. [10 marks] Design an algorithm based on sorting that can solve the problem in Question 2 in $O(n^2)$ -time.

To receive full marks, the running time of your algorithm must be $O(n^2)$. Otherwise, you can receive up to 5 marks if your algorithm runs in $O(n^2 \lg n)$ time, but no marks will be given if more time is needed.

You are not allowed to make assumptions on the precision and the possible range of values of the lengths recorded, except that each length is positive.